

ACT on AAP 2.7

Step-by-step runbook: templates, workflow with approval, schedules and Microsoft Teams reporting

Podman app health, fapolicyd analysis, triage | ACT 0.6.18 | September 2026

What this runbook is. Every step, in order, to run ACT from Red Hat Ansible Automation Platform 2.7: the credential types and credentials, the project and inventory, four job templates with their surveys, a workflow with an approval step, schedules, and Microsoft Teams reporting. It covers three jobs you asked for: **podman application health** (STIG Manager, nginx, MySQL, Keycloak: examine logs, restart when that is the right fix), **fapolicyd denial analysis** (what is blocked and the minimal safe allow), and **Teams reporting**.

About the menu labels

AAP 2.5, 2.6 and 2.7 share the same unified web interface. The labels in this runbook come from the AAP 2.5 documentation (the 2.7 pages were not retrievable when it was written), so an occasional label may be worded slightly differently in your 2.7 console. The objects and the order of the steps are the same.

Contents

Overview	3
Part A. Prepare (once)	4
Part B. Credential types and credentials	4
Part C. Project and inventory	6
Part D. Job templates	6
D1. "ACT - Podman app health"	7
D2. "ACT - fapolicyd denial analysis"	7
D3. "ACT - Triage (generic)"	8
D4. "ACT - Apply approved fixes"	8
D5. Adding a survey question	8
D6. Test each template	8
Part E. Workflow with an approval step	9
Part F. Schedules	10
Part G. Microsoft Teams	10
Part H. Verify and operate	12
Appendix. Notifier message bodies	14

Overview

You will build four job templates and one workflow. The playbooks live in the ACT-Linux repository under [ansible/playbooks/](#).

AAP object	type	playbook	purpose
ACT - Podman app health	Job template	podman_app_health.yml	Check containers; ACT diagnoses and, if allowed, restarts
ACT - fapolicyd denial analysis	Job template	fapolicyd_denials.yml	What fapolicyd blocked, and the minimal safe allow
ACT - Triage (generic)	Job template	act_triage.yml	Failed units, disk, journal errors
ACT - Apply approved fixes	Job template	act_apply_approved.yml	Applies exactly the fixes a person approved
ACT - Podman health + approve + fix	Workflow	(the templates above)	Check, approval, apply, verify

set_stats carries each host's proposed_commands from the first job to the last

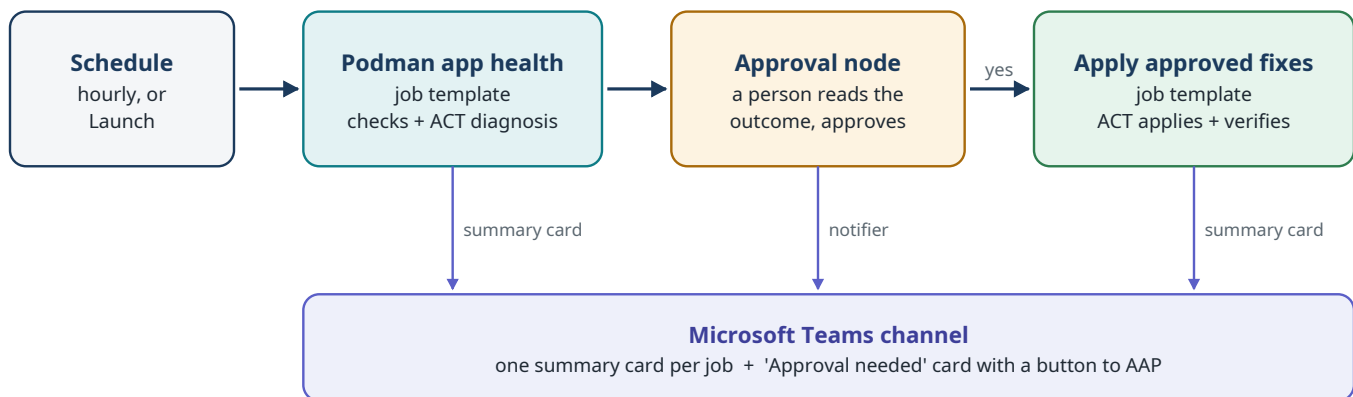


Figure 1. The workflow and what it posts to Teams

Firewall: execution node -> hosts (SSH), hosts -> model endpoint (HTTPS), execution node -> Teams webhook (HTTPS)

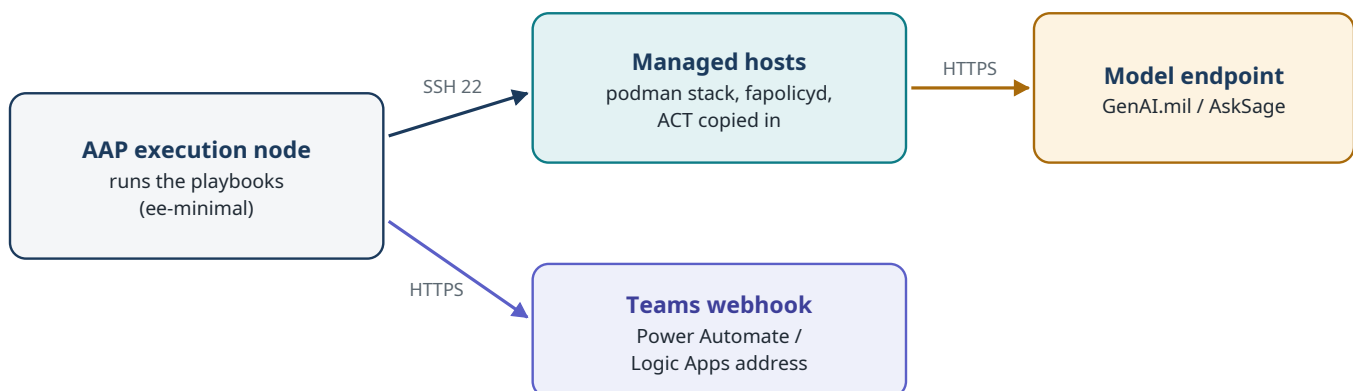


Figure 2. Network connections to allow

Part A. Prepare (once)

A1. Prepare the managed hosts

1. **Python 3.8 or newer** on every host. RHEL 9: the default `python3` is fine. RHEL 8: `dnf install -y python3.11` (installs next to the default; the role finds it).
2. **Network:** each host must reach the model endpoint over HTTPS. Test from one host, for example `curl -sI https://api.genai.mil` (or your AskSage address).
3. **Privilege:** the SSH user of your Machine credential needs `sudo`. The playbooks use `become: true`: the `journal`, `ausearch` and restarts need root.
4. **Rootless podman only:** if the containers run under a service account instead of root, that account needs lingering: `loginctl enable-linger <account>`. (Supported by the playbook, not yet tested in a real deployment.)

A2. Put the content where AAP can read it

AAP pulls playbooks from Git: use the ACT-Linux repository or an internal mirror of it. A private repository needs a **Source Control** credential (token or deploy key, created in B3). Pin the project to a **release tag** rather than `main`, so repository changes never reach production automation until you move the tag.

A3. Make AAP's links point to the right address

Notification and Teams buttons link back to AAP:

1. From the navigation panel, select **Settings › System**.
2. Click **Edit**.
3. In **Base URL of the service**, enter the address users open, e.g. `https://aap.example.mil`.
4. Click **Save**.

A4. Execution environment

The roles use only `ansible.builtin` modules, so the minimal execution environment (`ee-minimal`) is enough; `ee-supported` works too. No extra collections are needed.

Part B. Credential types and credentials

B1. Credential type "ACT model key"

1. From the navigation panel, select **Automation Execution › Infrastructure › Credential Types**.
2. Click **Create credential type**.
3. **Name:** `ACT model key`. **Description:** `GenAI / AskSage API key for ACT`.
4. Paste the **Input configuration** and **Injector configuration** below.
5. Click **Create credential type**.

Input configuration

```
fields:
- id: genai_key
  type: string
  label: GenAI API key
  secret: true
- id: asksage_key
  type: string
  label: AskSage API key
  secret: true
```

Injector configuration

```
env:
  GENAI_KEY: "{{ genai_key }}"
  ASKSAGE_API_KEY: "{{ asksage_key }}"
```

The key exists only inside the job's execution environment. The role hands it to ACT on each host through the task environment with `no_log`: it never shows in job output.

B2. Credential type "Teams webhook"

1. **Automation Execution › Infrastructure › Credential Types › Create credential type.**
2. **Name:** Teams webhook.
3. Paste the configurations below, then click **Create credential type.**

Input configuration

```
fields:
- id: webhook_url
  type: string
  label: Teams Workflows webhook URL
  secret: true
required: [webhook_url]
```

Injector configuration

```
env:
  TEAMS_WEBHOOK_URL: "{{ webhook_url }}"
```

The webhook URL contains a secret signature (anyone with it can post to the channel), which is why it is a secret credential field and not a variable.

B3. Credentials

From the navigation panel, select **Automation Execution › Infrastructure › Credentials**, then **Create credential** for each, choose your **Organization**, and click **Create credential**:

name	credential type	fields
ACT model key	ACT model key	the GenAI and/or AskSage key
Teams - ops channel	Teams webhook	the URL from G1 (create this after G1)
Linux ssh (sudo)	Machine	Username, SSH Private Key, Privilege Escalation Method <code>sudo</code> , Privilege Escalation Username <code>root</code> (and password if your sudo needs one)

name	credential type	fields
ACT-Linux git	Source Control	private repositories only: username + token, or an SSH key

Part C. Project and inventory

C1. Project

1. **Automation Execution** › **Projects** › **Create project**.
2. **Name** `ACT-Linux`; **Organization** yours; **Execution environment** `ee-minimal`.
3. **Source control type** `Git`; **Source control URL** the ACT-Linux repository; **Source control branch/tag/commit** the release tag; **Source control credential** `ACT-Linux git` if private.
4. Tick **Clean**. Leave **Update revision on launch** off when you pin a tag.
5. Click **Create project** and wait for the sync status **Successful**.

To move to a new ACT release later: edit the project, change the tag, save, and sync.

C2. Inventory

1. **Automation Execution** › **Infrastructure** › **Inventories** › **Create inventory**.
2. **Name** `Linux servers`; **Organization** yours. Click **Create inventory**.
3. **Hosts** tab: **Create host** for each server.
4. **Groups** tab: create `stigman_hosts` (hosts that run the STIG Manager containers) and `app_servers` (hosts to check for fapolicyd denials).
5. Per-host settings go in the host's **Variables**, e.g. for rootless containers: `podman_user: stigman`.

Part D. Job templates

Every template is created the same way: **Automation Execution** › **Templates**, then from the **Create template** list select **Create job template**, fill in the fields, and click **Create job template**. These fields are the same for all four:

field	value
Job type	Run
Inventory	<code>Linux servers</code>
Project	<code>ACT-Linux</code>
Execution environment	<code>ee-minimal</code>
Credentials	<code>Linux ssh (sudo)</code> , <code>ACT model key</code> , <code>Teams - ops channel</code>
Limit	leave empty and tick Prompt on launch : workflows and schedules can then pass a host group
Timeout	<code>3600</code>
Enable concurrent jobs	off : two runs on the same hosts must not overlap

D1. "ACT - Podman app health"

Playbook `ansible/playbooks/podman_app_health.yml`. **Variables** (edit container names, units and URLs to match your deployment):

Variables

```
target: stigman_hosts
act_report_teams_aap_url: https://aap.example.mil
podman_apps:
- name: mysql
  container: mysql
  unit: "" # e.g. mysql.service (quadlet) or container-mysql.service
  probe_cmd: "podman exec mysql mysqladmin ping -h 127.0.0.1" # MariaDB: mariadb-admin
  depends_on: []
- name: keycloak
  container: keycloak
  unit: ""
  probe_url: "http://127.0.0.1:8080/realms/master"
  depends_on: []
- name: stigman
  container: stigman
  unit: ""
  probe_url: "http://127.0.0.1:54000/api/op/configuration"
  depends_on: [mysql, keycloak]
- name: nginx
  container: nginx
  unit: ""
  probe_url: "https://127.0.0.1/"
  depends_on: [stigman, keycloak]
```

The probe URLs are examples: use each application's real health or login URL. **Survey** (D5): *Multiple Choice (single select)*, answer variable `podman_allow_restart`, choices `false` and `true`, default `false`.

What it does

Checks that each container exists and is running, its healthcheck, its restart count, the number of error-looking lines in the last 30 minutes of its logs, and each probe URL or command. ACT runs only on hosts where something failed.

ACT is told the dependency order (nginx > stigman > MySQL, Keycloak), so it fixes the database before blaming STIG Manager. With `podman_allow_restart: true` it may restart exactly these containers (or their systemd units), one at a time, dependencies first, and must verify each with `podman inspect`. It never restarts for configuration, credential, certificate or disk problems: it reports those.

D2. "ACT - fapolicyd denial analysis"

Playbook `ansible/playbooks/fapolicyd_denials.yml`. **Variables**:

Variables

```
target: app_servers
act_report_teams_aap_url: https://aap.example.mil
```

Survey: *Multiple Choice (single select)*, answer variable `fapolicyd_since`, choices `recent`, `today`, `yesterday`, `this-week`, default `today`.

What it does

Skips hosts without `fafolicyd` and reports if `auditd` is not running: denials are read from the audit log, so they need `auditd` and deny rules that audit (`deny_audit`, as in the RHEL default rules). Runs `ausearch -m FANOTIFY` for denials in the chosen window. If there are any, it collects the evidence: `fafolicyd` settings, the numbered rule list (`fafolicyd-cli --list`), the rule and trust files, the denials, and for each denied file its checksum, owning package, trust status and SELinux label.

ACT explains, per blocked access, **what** was blocked, **which rule** blocked it, whether it looks **legitimate**, and the **minimal fix** (a trust entry for that exact file, otherwise a narrow rule). ACT runs **no** commands in this job and nothing is ever applied.

Security-control changes

Trust entries and `fafolicyd` rules change a security control. They need your security approval (for example the ISSO) before anyone applies them. Never put this template in front of the apply-approved step.

D3. "ACT - Triage (generic)"

Playbook `ansible/playbooks/act_triage.yml`. **Variables:** `act_triage_journal_error_threshold: 100` and `act_report_teams_aap_url`. Optional **survey:** *Textarea*, answer variable `act_triage_allow_lines`, "Fix commands ACT may run unattended, one regular expression per line", not required, default empty.

D4. "ACT - Apply approved fixes"

Playbook `ansible/playbooks/act_apply_approved.yml`. **Variables:** `act_report_teams_aap_url`. No survey. It only makes sense inside a workflow (Part E): it reads the previous job's results (`act_triage`) and applies exactly the approvable proposed commands.

D5. Adding a survey question

1. Open the template (**Automation Execution > Templates** > the template name).
2. Select the **Survey** tab and click **Create survey question**.
3. Fill in **Question**, **Answer variable name** (exactly as listed above), **Answer type**, **Required**, the choices (one per line) for multiple-choice questions, and **Default answer**.
4. Click **Create question**.
5. Turn the survey **on** with the toggle on the **Survey** tab.

D6. Test each template

1. **Automation Execution > Templates**, click **Launch template** (the rocket) next to it.
2. Keep the survey defaults and give a **Limit** of one test host.
3. In the output look for: the `... | summary` task (what the checks found), `ACT | outcome` per host (`status`, `summary`, `proposed_fixes`, `manual_fixes`), and `Teams | post the summary card`.

`ASYNCR FAILED` on the ACT run task is expected when ACT exits 4 (`needs_approval`); the outcome comes from ACT's result file.

Part E. Workflow with an approval step

E1. Create the workflow template

1. **Automation Execution > Templates**; from **Create template** select **Create workflow job template**.
2. **Name** `ACT - Podman health + approve + fix`; **Organization** yours; **Inventory** `Linux servers`; **Limit**: tick **Prompt on launch**. **Variables**: `act_triage_fail_on: []` (one host in error must not stop the approval for the others; errors still show in Teams and the job output). Leave **Enable concurrent jobs** off.
3. Click **Create workflow job template**. The **workflow visualizer** opens.

E2. Build the graph in the visualizer

1. Click **Add step**. **Node type** *Job template*, select **ACT - Podman app health**, click **Finish**. The first node always runs.
2. Hover over that node and choose **Add step and link**. **Node type** *Approval*: **Name** `Apply ACT's proposed restarts?`, **Description** `Read the triage job output first (ACT | outcome per host).`, **Timeout** e.g. 4 hours (unanswered requests are then denied). Run condition **Run on success**. Click **Finish**.
3. Hover over the approval node, **Add step and link**. **Node type** *Job template*, select **ACT - Apply approved fixes**, run condition **Run on success**. Click **Finish**.
4. Click **Save**. (Close without **Save** discards the whole graph.)

The graph

```
[ACT - Podman app health] --(on success)--> [Approval] --(approved)--> [ACT - Apply approved fixes]
```

How the pieces connect: the podman job publishes each host's result with `set_stats (act_triage: {host: {status, summary, proposed_commands, manual_fixes}})`, and AAP passes it to the later nodes. The apply job turns each approved command into an exact-match allow pattern; ACT re-checks the problem, runs it, and verifies. `manual_fixes` are never applied. The same pattern works for **ACT - Triage (generic)**.

E3. When to run this workflow

Launch it on demand - do not schedule it

An approval node follows every successful check run, so a scheduled workflow would raise an approval request every hour even when every host is healthy, and unanswered requests would pile up (concurrent runs are off).

Instead, schedule the **ACT - Podman app health** job template (Part F): it reports to Teams. When a card shows **needs approval**, launch this workflow limited to those hosts, read the outcome, and approve.

E4. Who can approve

A person can approve if they can execute the workflow, are an organization administrator, or have the **Approve** permission on this workflow template. Give your approvers (as a team) the approve role on the workflow template, rather than execute rights on everything.

E5. Test the workflow

1. Launch the workflow with **Limit** = one test host.
2. When it pauses, the workflow job shows the approval node as pending (with Part G, Teams also gets an "Approval needed" card with a button).
3. Open the **ACT - Podman app health** job, read each host's **ACT | outcome**, then **Approve** or **Deny** the approval node.
4. After approval, open **ACT - Apply approved fixes**: each host shows **status: completed** and the restart with **approval: pre_approved**.

Limits in workflows

A workflow's limit reaches a job template only when that job template has **Prompt on launch** ticked for **Limit**. That is why Part D ticks it on every template.

Part F. Schedules

F1. Hourly podman health

Schedule the **job template**, not the approval workflow (see E3).

1. Open **ACT - Podman app health** and select the **Schedules** tab.
2. Click **Create schedule**.
3. **Schedule name** **Hourly**; **Start date/time** a few minutes ahead; **Time zone** yours. Click **Next**.
4. **Define rules: Frequency** Hourly, **Interval** 1, **Minutes of the hour** 5. Click **Save rule**, then **Next**.
5. **Exceptions**: none. Click **Next**, review, and save the schedule.

F2. Daily fapolicyd report

On **ACT - fapolicyd denial analysis**: a schedule with **Frequency** Daily at 06:00, answering the survey prompt with **fapolicyd_since: yesterday**.

AAP stores schedules in UTC, so a daily time can shift by an hour at daylight saving changes. Healthy hosts cost nothing: the model is only called where a check tripped. To let the hourly run restart failed containers without waiting for a person, answer **podman_allow_restart: true** on the schedule's survey prompt, once you trust the diagnoses.

Part G. Microsoft Teams

Microsoft retired the old Teams "Incoming Webhook" (Office 365) connectors in 2026. Posting to a channel now uses a **Teams Workflows** webhook, which accepts an Adaptive Card.

Check before you build on it

GCC High and DoD tenants may not offer the Workflows webhook trigger, or may use government endpoints. Ask your Microsoft 365 administrator first. The AAP execution nodes also need outbound HTTPS to the webhook address.

G1. Create the webhook in Teams

1. In Teams, open the channel (e.g. *Ops - Automation*), click **⋮ > Workflows**.

2. Choose **Post to a channel when a webhook request is received**.
3. Name it (e.g. **AAP ACT reports**), confirm the team and channel, and click **Add workflow**.
4. Copy the URL it shows (a Power Automate / Logic Apps address with **sig=** in it) into the **Teams - ops channel** credential (B3).

G2. Summary card after every run (built in)

Every playbook ends with the **act_report_teams** role. With the Teams credential attached to the template it posts **one card per run**, only when something needs attention (**act_report_teams_when: always** posts every run):

- the job template name and the time;
- counts: healthy, fixed, needs approval, stopped/error, unreachable;
- per host (up to 12): status, the checks that tripped, ACT's summary (shortened), proposed approvable commands, and fixes to apply by hand;
- an **Open job in AAP** button when **act_report_teams_aap_url** is set.

The card is posted **before** any host is marked failed, so it arrives even when every host errors (for example when the model endpoint is down). Hosts whose status is in **act_triage_fail_on** (default **error**) are failed afterwards, so AAP failure notifications still fire.

G3. Approval requests and failures (AAP notifier)

1. From the navigation panel, select **Automation Execution > Administration > Notifiers**.
2. Click **Create notifier**. Name **Teams - approvals**; **Organization** yours; **Type** *Webhook*.
3. **Target URL**: the Teams webhook URL. **HTTP Method** *POST*. Leave username, password and headers empty; keep SSL verification on.
4. Turn on **Customize messages** and paste the JSON bodies from the appendix into **Workflow pending message body**, **Workflow approved body**, **Workflow denied message body**, **Workflow timed out message body**, and **Error message body**. Save.
5. Open the workflow > **Notifications** tab > turn on **Approval** and **Failure** for **Teams - approvals**.
6. Also turn on **Failure** for **Teams - approvals** on the **scheduled job templates** (podman health, fapolicyd), so a failed scheduled run is reported.

Approvers then get "Approval needed: Apply ACT's proposed restarts?" with a **Review and approve in AAP** button that opens the pending approval.

G4. If Workflows is not available

- Use an **Email** notifier to a distribution list, or to the channel's email address if your tenant allows channel email.
- The summary is always in the AAP job output (**ACT | outcome**).

Part H. Verify and operate

H1. First-week checklist

- Each job template launched once by hand against one test host (D6).
- The workflow approved once and denied once (E4).
- A Teams summary card and an approval card arrived (G2, G3).
- Thresholds tuned so healthy hosts do not trip checks: every tripped check is a model call.
- `podman_allow_restart` stays `false` until the diagnoses have been right for a while.

H2. Where to look

question	where
What did the checks find?	job output, task <code>...</code> <code>summary</code>
What did ACT conclude?	job output, task <code>ACT</code> <code>outcome</code> , and the Teams card
What did ACT run?	ACT's result file <code>commands</code> (the fleet report shows it as a table)
What is waiting for approval?	the workflow job's approval node; the Teams approval card
What changed on a host?	<code>changed</code> in the outcome; <code>pre_approved</code> commands in the report

H3. Troubleshooting

symptom	cause	fix
error , "No API key is configured"	ACT model key credential not attached	attach it; for AskSage also set act_triage_env : {ACT_PROVIDER: asksage}
"ACT needs Python 3.8+"	RHEL 8 default Python	dnf install -y python3.11
error , network or TLS error	host cannot reach the model endpoint	open egress; set GENAI_CA in act_triage_env for an internal CA
Podman checks see no containers	containers are rootless	set podman_user (host variable) to the owning account
An expected restart was not done	podman_allow_restart false, or the container has a unit	set the survey answer; fill unit for systemd-managed containers
Teams: nothing posted	nothing needed attention, or no credential	act_report_teams_when : always to test; attach the Teams credential
Teams task fails	webhook URL wrong or expired, or no egress	recreate the Teams workflow; allow HTTPS from the execution nodes
Buttons open the wrong address	Base URL not set	A3
fapolicyd: "no denials" while something is blocked	auditd not running, or deny rules that do not audit	start auditd; deny rules must be deny_audit (RHEL defaults), not deny / deny_syslog
The workflow never reaches the approval	a host ended in error and failed the check job	act_triage_fail_on : [] in the workflow's variables (E1)

Appendix. Notifier message bodies

Notifier **Type** *Webhook*, **Customize messages** on. AAP fills in the `{{ }}` values. These bodies are valid JSON with sample values substituted; the AAP rendering itself has not yet been tested against a live AAP 2.7. For copy-paste, use `docs/AAP_2.7_RUNBOOK.md` in the repository rather than this PDF.

Workflow pending message body

```
{
  "type": "message",
  "attachments": [
    {
      "contentType": "application/vnd.microsoft.card.adaptive",
      "contentUrl": null,
      "content": {
        "$schema": "http://adaptivecards.io/schemas/adaptive-card.json",
        "type": "AdaptiveCard",
        "version": "1.4",
        "msteams": {
          "width": "Full"
        },
        "body": [
          {
            "type": "TextBlock",
            "size": "Medium",
            "weight": "Bolder",
            "wrap": true,
            "text": "Approval needed: {{ approval_node_name }}"
          },
          {
            "type": "TextBlock",
            "wrap": true,
            "text": "{{ job_friendly_name }} #{{ job.id }}: read the triage output, then decide."
          }
        ],
        "actions": [
          {
            "type": "Action.OpenUrl",
            "title": "Review and approve in AAP",
            "url": "{{ workflow_url }}"
          }
        ]
      }
    }
  ]
}
```

Workflow approved / denied / timed out body (same body for all three)

```
{
  "type": "message",
  "attachments": [
    {
      "contentType": "application/vnd.microsoft.card.adaptive",
      "contentUrl": null,
      "content": {
        "$schema": "http://adaptivecards.io/schemas/adaptive-card.json",
        "type": "AdaptiveCard",
        "version": "1.4",
        "body": [
          {
            "type": "TextBlock",
            "weight": "Bolder",
            "wrap": true,
            "text": "{{ approval_node_name }}: {{ approval_status }}"
          },
          {
            "type": "TextBlock",
            "wrap": true,
            "text": "{{ job_friendly_name }} #{{ job.id }}"
          }
        ],
        "actions": [
          {
            "type": "Action.OpenUrl",
            "title": "Open in AAP",
            "url": "{{ workflow_url }}"
          }
        ]
      }
    }
  ]
}
```

Error message body

```
{
  "type": "message",
  "attachments": [
    {
      "contentType": "application/vnd.microsoft.card.adaptive",
      "contentUrl": null,
      "content": {
        "$schema": "http://adaptivecards.io/schemas/adaptive-card.json",
        "type": "AdaptiveCard",
        "version": "1.4",
        "body": [
          {
            "type": "TextBlock",
            "weight": "Bolder",
            "color": "Attention",
            "wrap": true,
            "text": "FAILED: {{ job_friendly_name }} #{{ job.id }}"
          },
          {
            "type": "TextBlock",
            "wrap": true,
            "text": "Status: {{ job.status }}"
          }
        ],
        "actions": [
          {
            "type": "Action.OpenUrl",
            "title": "Open the job",
            "url": "{{ url }}"
          }
        ]
      }
    }
  ]
}
```

Rootless podman host variable

Host variables

```
podman_user: stigman          # the account that owns the containers (loginctl enable-linger stigman)
```

With `podman_user` set, the playbooks run the checks and ACT as that account, install ACT under its home directory, set `XDG_RUNTIME_DIR`, and allow `systemctl --user restart UNIT` for containers that have units.